ECSE 222, Digital Logic

VHDL 2:

Design and Simulation of Digital Circuits

Fall 2023
University of McGill

Presented by:

Nara Yun (Student #: 261115268)

Karen Chen Lai (Student #: 261107674)

Instructors: Prof. Boris Vaisband

Teaching Assistant: Maninder Bir Singh Gulshan

Date: October 3rd, 2023

Executive Summary

This experiment consists of three parts in which we executed and learnt the creation of circuit, use of the testbench writer tool in Quartus, and writings of concurrent statements.

The first part is composed of using CAD tools to create a XNOR circuit with 4-bit inputs using a schematic diagram in Quartus and simulation of the HDL in Modelsim. We converted our comparator schematic into a VHDL file by creating a HDL design file after we have successfully compiled our design circuit, we also created a testbench for the circuit through the Test Bench Writer in Quartus. After that, we created a project in ModelSim Altera using these two files to simulate the waveform of the design.

The second part consists of implementation of 2-to-1 MUX. We wrote the entities, behavioral and structural architectures, and the corresponding testbenches for the 2-to-1 MUX. Then we wrote an exhaustive test to evaluate our VHDL descriptions through the RTL simulation.

In the last part, we wrote a 4-bit circular barrel shifter in VHDL using both structural and behavioral styles by implementing the 2-to-1 MUX that we created in the second part. Moreover, we discovered the use of nested for loops to perform an exhaustive test in the test bench for our VHDL descriptions of the 4-bit circular barrel shifter.

Questions

1. Briefly explain your VHDL code implementation of all circuits.

```
-- Copyright (C) 2018  Intel Corporation. All rights reserved.
-- Your use of Intel Corporation's design tools, logic functions
-- and other software and tools, and its AMPP partner logic
-- functions, and any output files from any of the foregoing
-- (including device programming or simulation files), and any
-- associated documentation or information are expressly subject
-- to the terms and conditions of the Intel Program License
-- Subscription Agreement, the Intel Quartus Prime License Agreement,
-- the Intel FPGA IP License Agreement, or other applicable license
-- agreement, including, without limitation, that your use is for
-- the sole purpose of programming logic devices manufactured by
-- Intel and sold by Intel or its authorized distributors.  Please
-- refer to the applicable agreement for further details.

-- ***************************************************************************
-- This file contains a Vhdl test bench template that is freely editable to
-- suit user's needs .Comments are provided in each section to help the user
-- fill out necessary details.
-- ***************************************************************************
-- Generated on "09/25/2023 14:56:17"

-- Vhdl Test Bench template for design  :  Nara_Yun_comp
--
-- Simulation tool : ModelSim-Altera (VHDL)
--

LIBRARY ieee;
USE ieee.std_logic_1164.all;
use ieee.numeric_std.all;

ENTITY Nara_Yun_comp_vhd_tst IS
END Nara_Yun_comp_vhd_tst;
ARCHITECTURE Nara_Yun_comp_arch OF Nara_Yun_comp_vhd_tst IS
-- constants
-- signals
SIGNAL A : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL AeqB : STD_LOGIC;
SIGNAL B : STD_LOGIC_VECTOR(3 DOWNTO 0);
COMPONENT Nara_Yun_comp
    PORT (
    A : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
    AeqB : OUT STD_LOGIC;
    B : IN STD_LOGIC_VECTOR(3 DOWNTO 0)
    );
END COMPONENT;
BEGIN
    i1 : Nara_Yun_comp
    PORT MAP (
-- list connections between master ports and signals
    A => A,
    AeqB => AeqB,
    B => B
    );

generate_test: PROCESS
BEGIN
    for i in 0 to 16 loop
    A <= std_logic_vector(to_unsigned(i,4));
    for j in 0 to 16 loop
        B <= std_logic_vector(to_unsigned(j,4));

        wait for 10 ns;
    end loop;
    end loop;
WAIT;
END PROCESS generate_test;
END Nara_Yun_comp_arch;

```

For the first part of the experiment, Figure 1 and Figure 2 are the VHDL descriptions and test bench converted by Quartus from analysing our 4 XNOR gates circuit. These two files describe 4-bit inputs A(0), A(1), A(2), A(3) and B(0), B(1), B(2), B(3) and a 1-bit output, AeqB. Where it results 1 when two inputs have the same values, otherwise it shows up 0.

Both the behavioral and structural architectures are applying for loops to iterate all possible values for A and B of 4 bits.

Figure 1. VHDL descriptions of schematic 4-XNOR circuit

```
-- Copyright (C) 2018  Intel Corporation. All rights reserved.
-- Your use of Intel Corporation's design tools, logic functions
-- and other software and tools, and its AMPP partner logic
-- functions, and any output files from any of the foregoing
-- (including device programming or simulation files), and any
-- associated documentation or information are expressly subject
-- to the terms and conditions of the Intel Program License
-- Subscription Agreement, the Intel Quartus Prime License Agreement,
-- the Intel FPGA IP License Agreement, or other applicable license
-- agreement, including, without limitation, that your use is for
-- the sole purpose of programming logic devices manufactured by
-- Intel and sold by Intel or its authorized distributors.  Please
-- refer to the applicable agreement for further details.

-- ***************************************************************************
-- This file contains a Vhdl test bench template that is freely editable to
-- suit user's needs .Comments are provided in each section to help the user
-- fill out necessary details.
-- ***************************************************************************
-- Generated on "09/25/2023 14:56:17"

-- Vhdl Test Bench template for design  :  Nara_Yun_comp
--
-- Simulation tool : ModelSim-Altera (VHDL)
--

LIBRARY ieee;
USE ieee.std_logic_1164.all;
use ieee.numeric_std.all;

ENTITY Nara_Yun_comp_vhd_tst IS
END Nara_Yun_comp_vhd_tst;
ARCHITECTURE Nara_Yun_comp_arch OF Nara_Yun_comp_vhd_tst IS
-- constants
-- signals
SIGNAL A : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL AeqB : STD_LOGIC;
SIGNAL B : STD_LOGIC_VECTOR(3 DOWNTO 0);
COMPONENT Nara_Yun_comp
        PORT (
        A : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
        AeqB : OUT STD_LOGIC;
        B : IN STD_LOGIC_VECTOR(3 DOWNTO 0)
        );
END COMPONENT;
BEGIN
        i1 : Nara_Yun_comp
        PORT MAP (
-- list connections between master ports and signals
        A => A,
        AeqB => AeqB,
        B => B
        );

generate_test: PROCESS
 BEGIN
        for i in 0 to 16 loop
        A <= std_logic_vector(to_unsigned(i,4));
        for j in 0 to 16 loop
                B <= std_logic_vector(to_unsigned(j,4));

                wait for 10 ns;
        end loop;
        end loop;
WAIT;
END PROCESS generate_test;
END Nara_Yun_comp_arch;
```

Figure 2. Testbench for schematic 4-XNOR circuit

NARA_YUN_MUX_structural.vhd | NARA_YUN_MUX_structural_tb.vh

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity NARA_YUN_MUX_behavioral is
    Port (A, B, S : IN STD_LOGIC;
          Y : out std_logic);
end NARA_YUN_MUX_behavioral;

architecture behavior of NARA_YUN_MUX_behavioral is
begin
    with S select Y <=
    A when '0',
    B when '1',
    'X' when others;
end behavior;
```

Figure 3. Behavioral architecture of 2-to-1 MUX

NARA_YUN_MUX_structural.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity NARA_YUN_MUX_structural is
    Port (A, B, S : IN STD_LOGIC;
          Y : out std_logic);
end NARA_YUN_MUX_structural;

architecture structural of NARA_YUN_MUX_structural is
begin
    Y <= ((not S) and A) or (B and S);
end structural;
```

Figure 4. Structural archiecture of 2-to-1 MUX

For the second part of this experiment, it consists of the VHDL implementation of a 2-to-1 MUX. Figure 3 and 4 illustrates the behavioral and structural architectures with the entities for the 2-to-1 MUX. Both of them describe the 2-to-1 multiplexer with 1 selector s which it chooses the output Y depending on the value of the selector. In this case, the output Y will be equal to A when s equals to ‘0’, it will be equal to B when s equals to ‘1’.

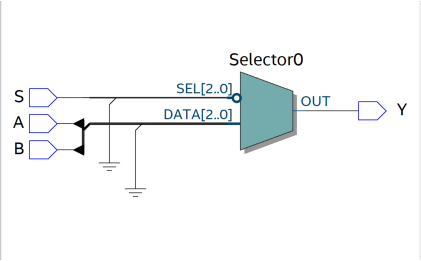

Figure 5 shows the 2-to-1 MUX schematic RTL view. Figure 6 and 7 are the testbench for behavioral and structural architectures respectively. Both are similar in applying nested for loops to iterate all possible cases with respect to selector, and inputs A and B.

Figure 5. 2-to-1 MUX schematic RTL view

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity NARA_YUN_MUX_behavioral_tb is

end NARA_YUN_MUX_behavioral_tb;

architecture tb of NARA_YUN_MUX_behavioral_tb is

    component NARA_YUN_MUX_behavioral
          port(A,B,S : in std_logic;
          Y: out std_logic);

    end component;

    signal A_in, B_in, S_in :std_logic;
    signal Y_out: std_logic;

begin

    dut : NARA_YUN_MUX_behavioral
    port map(A=>A_in,
             B => B_in,
             S => S_in,
             Y=> Y_out);

stimulate : process

begin
    S_in <= '0';
    A_in <= '0';
    B_in <= '0';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '0';
    A_in <= '0';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '0';
    A_in <= '1';
    B_in <= '0';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '0';
    A_in <= '1';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '1');

    S_in <= '1';
    A_in <= '0';
    B_in <= '0';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '1';
    A_in <= '0';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '1');

    S_in <= '1';
    A_in <= '1';
    B_in <= '0';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '1';
    A_in <= '1';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '1');

    assert false report "Test done." severity note;
    wait;
    end process;

end tb;
```

Figure 6. Testbench for behavioral archiecture of 2-to-1 MUX

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity NARA_YUN_MUX_structural_tb is

end NARA_YUN_MUX_structural_tb;

architecture tb of NARA_YUN_MUX_structural_tb is

    component NARA_YUN_MUX_structural
          port(A,B,S : in std_logic;
          Y: out std_logic);

    end component;

       function to_std_logic(i:integer) return std_logic is
       begin
          if i = 0 then return '0';
          elsif (i=1) then return '1';
          end if;
          return 'X';
       end function;

    signal A_in, B_in, S_in :std_logic;
    signal Y_out: std_logic;

begin

    dut : NARA_YUN_MUX_structural
    port map(A=>A_in,
             B => B_in,
             S => S_in,
             Y=> Y_out);

stimulate : process

begin
    S_in <= '0';
    A_in <= '0';
    B_in <= '0';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '0';
    A_in <= '0';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '0';
    A_in <= '1';
    B_in <= '0';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '0';
    A_in <= '1';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '1');

    S_in <= '1';
    A_in <= '0';
    B_in <= '0';
    wait for 1 ns;
    assert(Y_out = '0');

    S_in <= '1';
    A_in <= '0';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '1');

    S_in <= '1';
    A_in <= '1';
    B_in <= '1';
    wait for 1 ns;
    assert(Y_out = '1');

    assert false report "Test done." severity note;
    wait;
    end process;

end tb;
```

Figure 7. Testbench for structural archiecture of 2-to-1 MUX

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

--entity decla.
entity NARA_YUN_barrel_shifter_structural is

port(
    x: in std_logic_vector (3 downto 0);
     sel: in std_logic_vector (1 downto 0);
    y: out std_logic_vector (3 downto 0));
end NARA_YUN_barrel_shifter_structural;

--architecture decla.
architecture bs_structural of NARA_YUN_barrel_shifter_structural is

--intermediary signal decla.
signal out0, out1, out2, out3: std_logic;

--external component import
component mux

   port(
      a, b, s: in std_logic;
        y: out std_logic);
end component;

begin

   --level 1 Mux
    mux1: mux port map (a => x(0), b => x(2), s => sel(1), y=> out0);
    mux2: mux port map (a => x(1), b => x(3), s => sel(1), y=> out1);
    mux3: mux port map (a => x(2), b => x(0), s => sel(1), y=> out2);
    mux4: mux port map (a => x(3), b => x(1), s => sel(1), y=> out3);

    --level 2 Mux
    mux5: mux port map (a => out0, b => out3, s => sel(0), y=> y(0));
    mux6: mux port map (a => out1, b => out0, s => sel(0), y=> y(1));
    mux7: mux port map (a => out2, b => out1, s => sel(0), y=> y(2));
    mux8: mux port map (a => out3, b => out2, s => sel(0), y=> y(3));


end bs_structural;

```

Figure 8. Structural archiecture of 4-bit barrel shifter

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity mux is
port(
    a: in std_logic;
     b: in std_logic;
     s: in std_logic;
    y: out std_logic);
end mux;

architecture implementation of mux is
begin

   y <= (a and not s) or (b and s);

end implementation;

```

Figure 9. 2-to-1 MUX for 4-bit barrel shifter

NARA_YUN_barrel_shifter_behavioral.vhd

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

--entity
entity NARA_YUN_barrel_shifter_behavioral is

port(
    x: in std_logic_vector (3 downto 0);
     sel: in std_logic_vector (1 downto 0);
    y: out std_logic_vector (3 downto 0));
end NARA_YUN_barrel_shifter_behavioral;

--architecture
architecture bs_behavioral of NARA_YUN_barrel_shifter_behavioral is

begin

   --assignment
   with sel select

    y <= x(3) & x(2) & x(1) & x(0) when "00",
         x(2) & x(1) & x(0) & x(3) when "01",
         x(1) & x(0) & x(3) & x(2) when "10",
         x(0) & x(3) & x(2) & x(1) when others;

end bs_behavioral;
```

Figure 10. Behavioral archiecture of 4-bit barrel shifter

The last part of this experiment is the VHDL implementation of a 4-bit circular barrel shifter. Figure 8 and 10. Figure 10 demonstrates the barrel shifter behavioral architecture where it shows the 4 bit input X varies with regard to the 2 bit input sel. When the selector is ‘00’, the input X would not change; when the selector is ‘01’, the MSB of X will be shifted to LSB position; when the selector is ‘10’, the first two MSBs of X will be shifted to last two LSB positions; and when the selector is ‘11’, the first three MSBs of X will be shifted to LSB positions.

Figure 8 is the structural architecture of a 4-bit barrel shifter. Its architecture block describes the two levels MUX which uses two different selectors respectively. It utilizes the mux from the second part of this experiment, as shown as Fig. 9 and Fig. 4 are the same, eight times to implement the 4-bit circular barrel shifter. There are 4 mux with 2 inputs A and B, a selector s and an output Y in each mux levels.

Fig. 11 is the testbench for both behavioral and structural architectures. In this part, we applied the nested for loops to iterate input x and selector y for all 64 cases since there are 4 possible combinations for selector and 2^4 combinations of bits, so 4 * 16 = 64 cases.

NARA_YUN_barrel_shifter_behavioral.vhd | bs_tb.vhd | barrel_shifter_str

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3  use ieee.numeric_std.all
4
5  entity barrel_shifter_structural_testbench is
6  -- empty
7  end barrel_shifter_structural_testbench;
8
9  architecture tb of barrel_shifter_structural_testbench is
10 -- component declaration
11
12 component NARA_YUN_barrel_shifter_structural is
13 port(
14      x: in std_logic_vector (3 downto 0);
15      sel: in std_logic_vector (1 downto 0);
16      y: out std_logic_vector (3 downto 0));
17 end component;
18
19 --signal declaration
20 signal x_in, y_out: std_logic_vector (3 downto 0);
21 signal sel_in: std_logic_vector (1 downto 0);
22
23
24 begin
25    -- connect DUT
26    DUT: NARA_YUN_barrel_shifter_structural port map (x_in, sel_in, y_out);
27     process
28     begin
29         for i in 0 to 4 loop
30             sel_in <= std_logic_vector(to_unsigned(i, 2));
31             for j in 0 to 16 loop
32                x_in <= std_logic_vector(to_unsigned(j, 4));
33                wait for 10 ns;
34             end loop;
35          end loop;
36          report "Test complete";
37    wait;
38    end process;
39 end tb;
```

Figure 11. Testbenches for behavioral and structural archiectures of 4-bit barrel shifter

2. Report the number of pins and logic modules used to fit your designs on the FPGA board. These results can be obtained from the flow summary tab in the table of contents menu in Quartus.

	AeqB	2-to-1 MUX		4-bit circular barrel shifter	
	Schematic	Structural	Behavioral	Structural	Behavioral
Logic Utilization (in ALMs)	2/32070 (<1%)	1/32070 (<1%)	1/32070 (<1%)	5/32070 (<1%)	5/32070 (<1%)
Total pins	9/457 (<1%)	4/457 (<1%)	4/457 (<1%)	10/457	10/457

3. Show a representative simulation plot for the introductory testing example. You can simply include a snapshot from the waveform that you obtained from ModelSim. In order to fully capture all the signals from the waveform, you can adjust the display range using the magnifier icons.

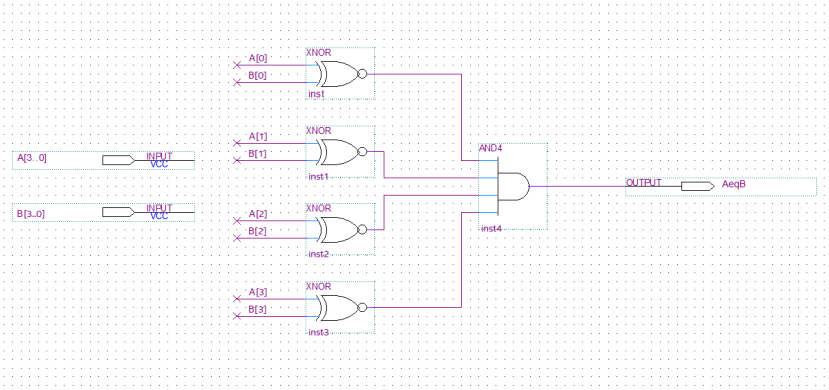

Figure 12. Designed circuit of 4 XNOR gates

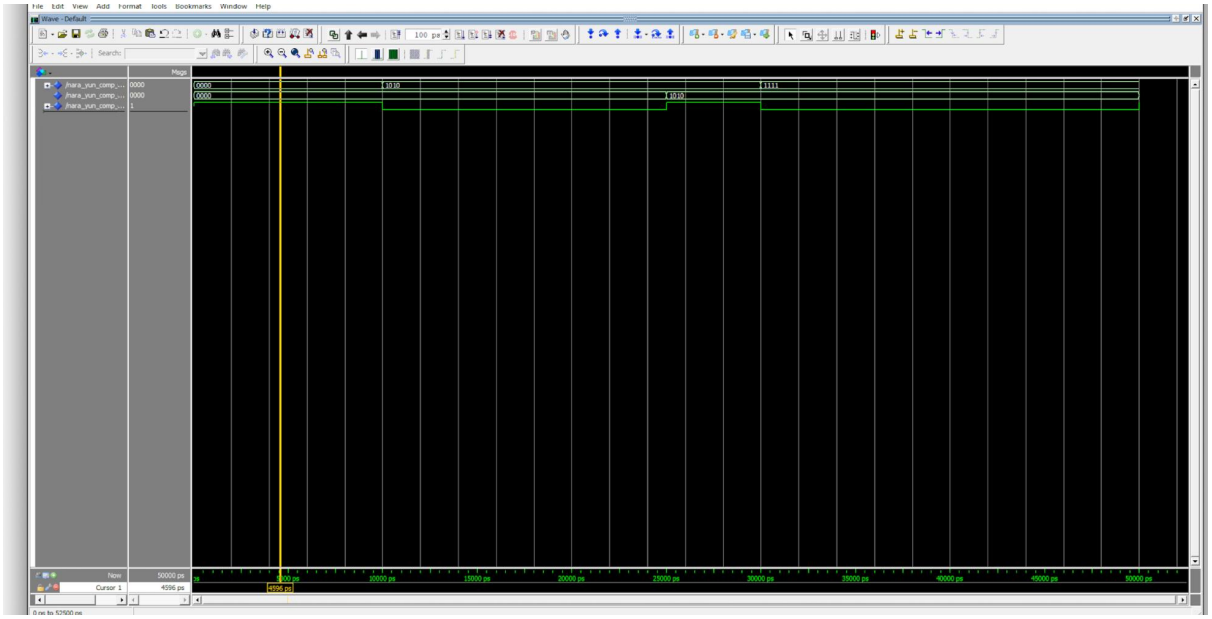

Figure 13. Simulation plot for the introductory test example

4. Show representative simulation plots for the exhaustive test.

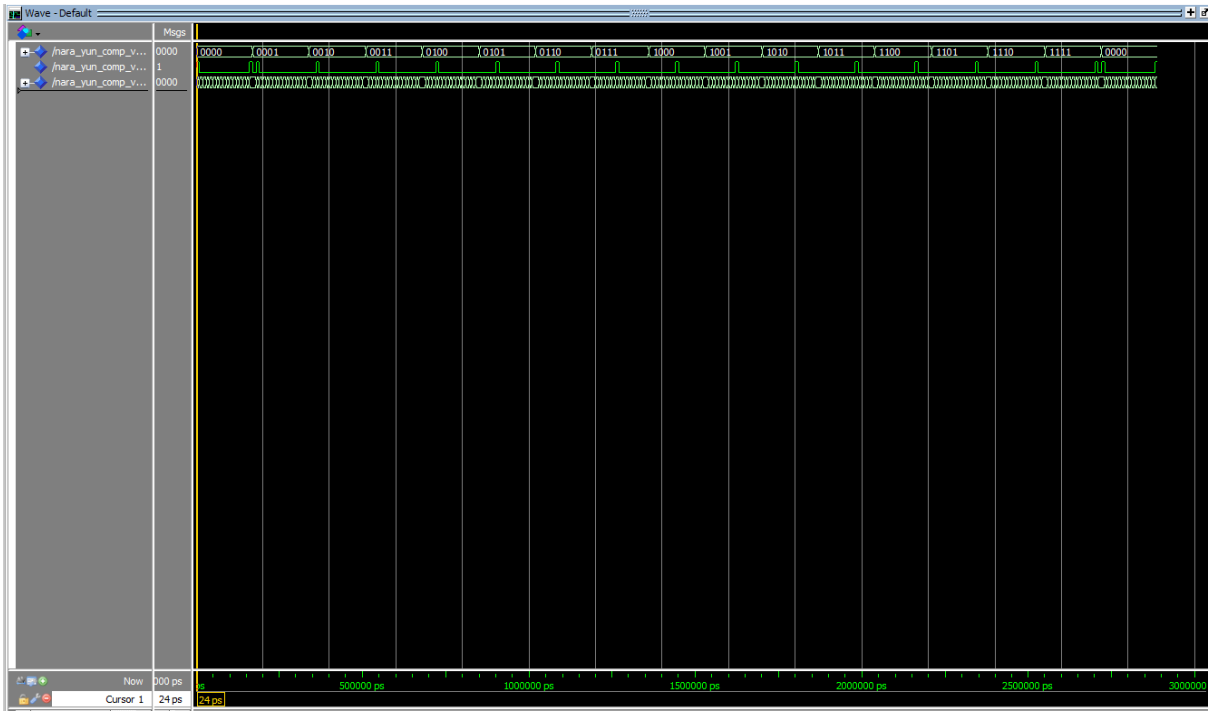

Figure 14. Simulation plots for the exhaustive test

5. Show representative simulation plots of the 2-to-1 MUX circuits for all the possible input values.

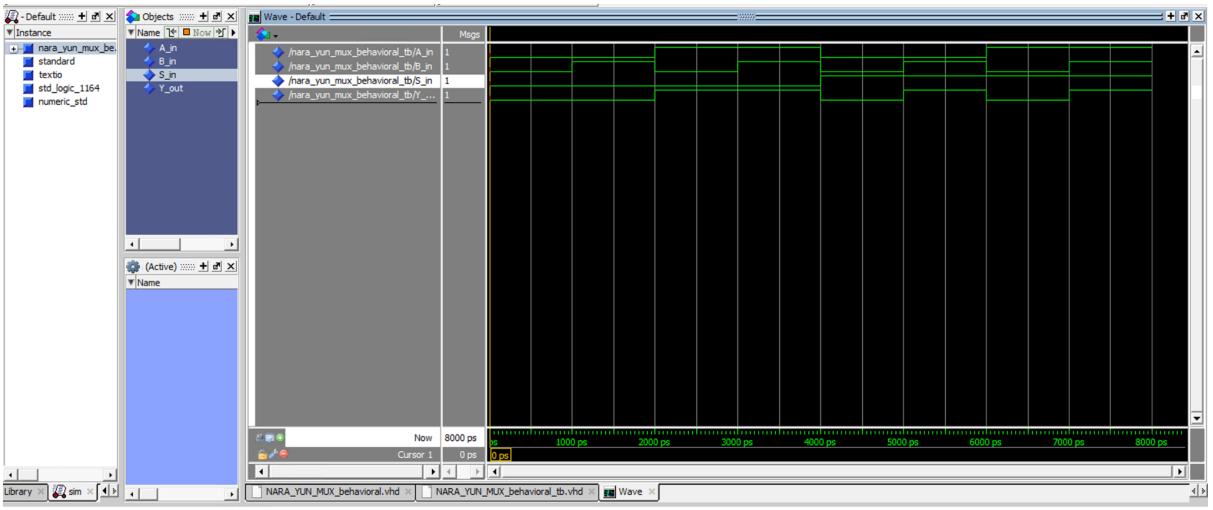

Figure 15. Simulation plots of the 2-to-1 MUX circuits

6. Show representative simulation plots of the 4-bit circular shift register circuits for a given input sequence, e.g., “1011” (X = “1011”), for all the possible shift amounts.

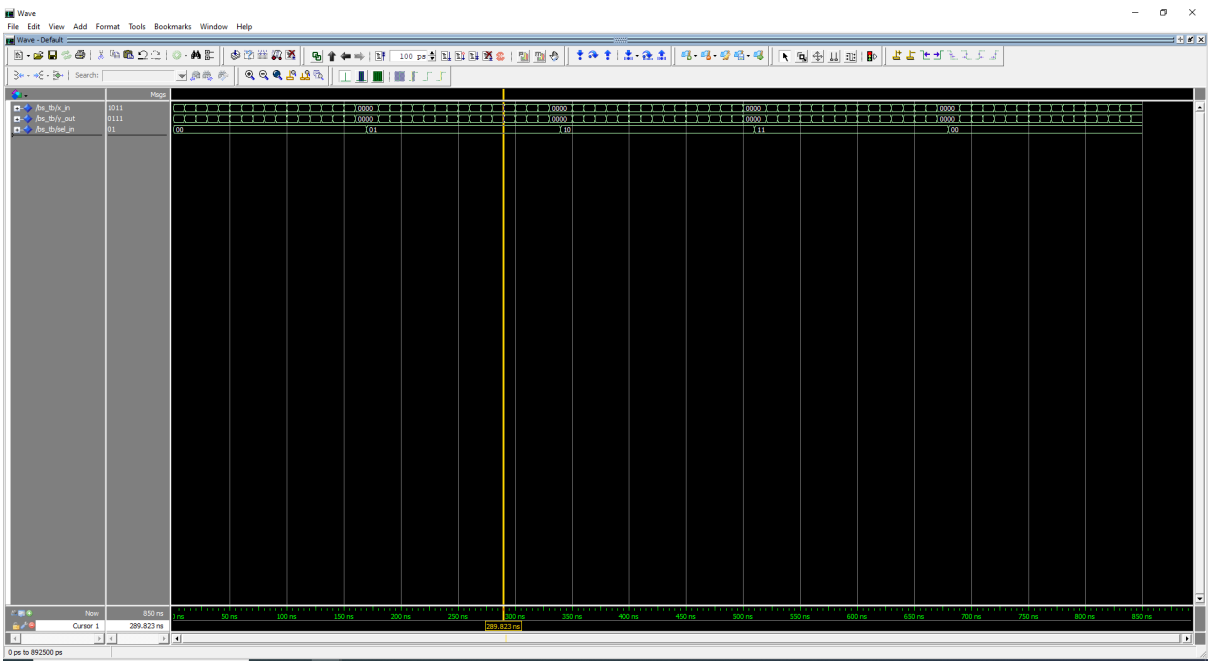

Figure 16. Simulation plots of the 4-bit circular shift

Explanation of the Results

From Figure 14, the AeqB circuit compares two 4-bit inputs, A and B, to determine if they are equal. It uses XNOR gates to calculate the equality of corresponding bits in A and B. Each pair of bits in A and B is compared using an XNOR gate to check if they are equal, and the results are combined using AND gates. If all bits are equal, the output AeqB is set to '1'; otherwise, it is '0'.

The 2-to-1 MUX (Multiplexer) circuit selects one of two inputs, A or B, based on the value of the selector signal S. When S is '0', the output Y is set to the value of A, and when S

is '1', the output Y is set to the value of B. From Figure 15, we can see that there are 8 possible outcomes. For example from Figure 15, when S_in is set to ‘0’, A_in is set to ‘0’ and B_in is set to ‘1’, we can see that the Y_out is set t0 ‘0’

Based on the sel signal, the 4-bit circular barrel shifter shifts the 4-bit input data X by a specified number of positions. It uses 2x1 multiplexers to perform circular shifting, with the sel signal determining the amount of shift. The circuit handles various shift scenarios, such as right shift, left shift, no shift, and circular shift, ensuring that bits that have been shifted out on one side reappear on the opposite side. The sel signal determines the number of positions to shift: '00' for no shift, '01' for right shift, '10' for left shift, and '11' for a circular shift. There are 64 possible output exits. For example from Figure 16, when x_in = “1011” and sel_in = “01” then y_out = “0111”.

Conclusions

Through this experiment, we learned how to:

- Create a design circuit using CAD tools.
- Convert our circuit design to VHDL file and its respective testbench file.
- Simulate designed circuit using ModelSim.
- Create VHDL descriptions and testbenches for a 2-to-1 multiplexer in behavioral and structural styles.
- Create VHDL descriptions and testbenches for a 4-bit circular barrel shifter including behavioral and structural styles.
- Fix the error syntaxes, including small mistakes, through interpreting and analyzing the RTL simulations.
- Enhance our observations and comprehensions on the architecture sections through the debugging process.